

MLOps Assignment 01 – Heart Disease Risk Prediction

- [MLOps Assignment 01 – End-to-End Heart Disease Risk Prediction](#)
 - [1. Project Overview](#)
 - [2. Dataset & Problem Statement](#)
 - [3. Setup & Installation](#)
 - [4. Exploratory Data Analysis \(EDA\)](#)
 - [5. Feature Engineering & Model Development](#)
 - [6. Experiment Tracking \(MLflow\)](#)
 - [7. Model Packaging & Reproducibility](#)
 - [8. CI/CD Pipeline & Automated Testing](#)
 - [9. Model Containerization](#)
 - [10. Production Deployment \(Kubernetes\)](#)
 - [11. Monitoring & Logging](#)
 - [12. Solution Architecture](#)
 - [13. Results Summary](#)
 - [14. Challenges & Lessons Learned](#)
 - [15. Future Improvements](#)
 - [16. Deliverables Mapping](#)
 - [17. References](#)
 - [Appendix A – Example API response](#)
 - [Appendix B – Video demo outline](#)

MLOps Assignment 01 – End-to-End Heart Disease Risk Prediction

Course: AIMLCZG523 – Machine Learning Operations

Assignment: Experimental Learning Assignment 01 (2026)

Dataset: Heart Disease UCI (Cleveland processed subset)

Student: AVINASH TIWARI [2024AC05513] **Repository:** <https://github.com/avinashtiwari18/heart-disease-mlops>

Deployed API: Local Docker (localhost:8000) and Kubernetes on Rancher Desktop (kubectl port-forward svc/heart-disease-api-svc 8000:80)

1. Project Overview

This project designs, trains, packages, and deploys a production-style machine learning classifier that predicts the presence of heart disease from patient clinical measurements. The solution follows modern MLOps practices:

1. Reproducible data acquisition and cleaning
2. Professional exploratory data analysis (EDA)
3. Feature engineering via an sklearn Pipeline / ColumnTransformer
4. Multi-model training with hyperparameter tuning and cross-validation
5. Experiment tracking with **MLflow**
6. Automated testing and **GitHub Actions** CI/CD
7. Containerized serving with **FastAPI** + **Docker**
8. Deployment to **Kubernetes** (Minikube / Docker Desktop)

9. Request logging and **Prometheus** metrics for monitoring

Business / clinical framing. Early risk stratification can support triage and follow-up decisions. The model is a decision-support classifier, not a diagnostic replacement for clinicians.

2. Dataset & Problem Statement

Item	Detail
Source	UCI Machine Learning Repository – Heart Disease
Primary file	processed.cleveland.data
Features	13 clinical attributes (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal)
Original label	num (0–4 severity)
Modelling label	Binary target: 0 if num == 0, else 1
Task	Supervised binary classification

Download instructions

```
python scripts/download_data.py
```

The script tries the official UCI Cleveland URL first, then a public mirror. It writes:

- data/raw/heart_disease_raw.csv
- data/cleaned/heart_disease_cleaned.csv

Missing values encoded as ? are parsed as NaN and median-imputed for the cleaned deliverable.

3. Setup & Installation

3.1 Prerequisites

- Python 3.11+
- Git
- Docker Desktop (for containerization)
- Minikube or Docker Desktop Kubernetes (for orchestration)
- Optional: kubectl, Helm, Grafana

3.2 Environment

```
cd AIMA/heart-disease-mlops
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/download_data.py
python -m src.eda
python -m src.train
pytest tests -v
uvicorn api.main:app --port 8000
```

All scripts are intended to run from a clean virtual environment using only requirements.txt.

4. Exploratory Data Analysis (EDA)

EDA artifacts are generated by `python -m src.eda` and `notebooks/01_eda.ipynb` into `artifacts/eda/`.

4.1 Missing value analysis

The Cleveland processed file historically contains missing values mainly in:

- `ca` (number of major vessels)
- `thal` (thalassemia status)

A missing-value bar chart and CSV report are saved under `artifacts/eda/`. Median imputation is used because the feature distributions are skewed and the missing rate is low relative to the sample size (~300 rows).

4.2 Class balance

The binary target is roughly balanced (near 46–54% depending on exact clean subset). This reduces the need for aggressive resampling; metrics beyond accuracy (precision, recall, F1, ROC-AUC) are still reported.

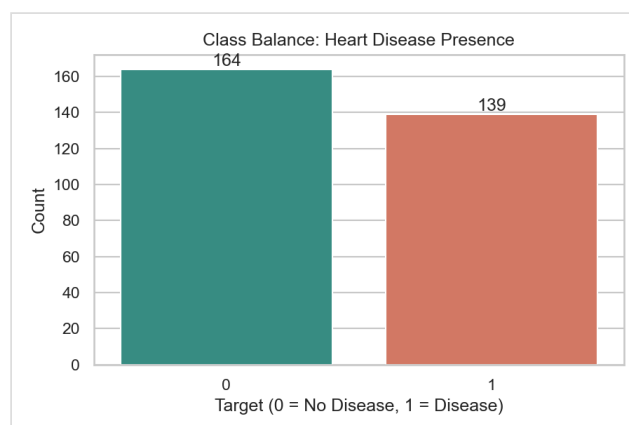
4.3 Distributions & relationships

- Histograms for `age`, `trestbps`, `chol`, `thalach`, `oldpeak` show clinical range spread and mild right-skew for cholesterol / ST depression.
- Boxplots vs target highlight that lower `thalach` and higher `oldpeak` tend to associate with disease presence.
- Correlation heatmap shows multi-collinearity among some ECG-related features; tree models tolerate this better than plain linear models, while Logistic Regression benefits from scaling + regularization.

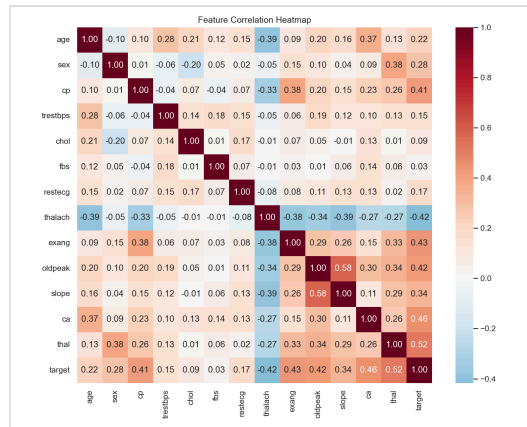
4.4 Modelling implications

- Scale continuous features for linear models.
- One-hot encode categorical clinical codes.
- Prefer ROC-AUC for model selection because it is threshold-robust.
- Keep a single serializable preprocessing+model pipeline for inference parity.

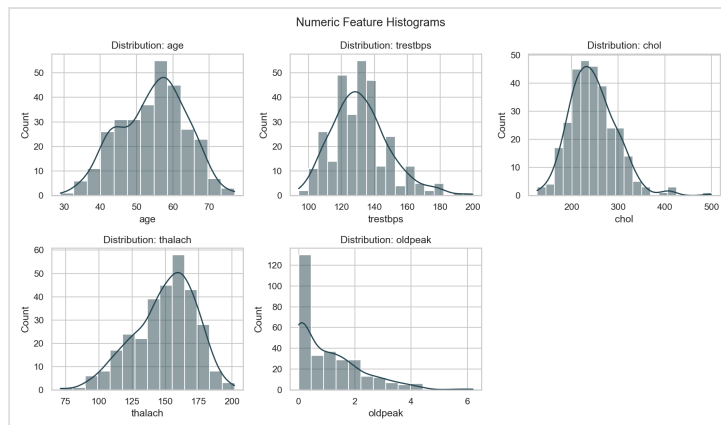
EDA figures



Class balance



Correlation heatmap



Numeric histograms

5. Feature Engineering & Model Development

5.1 Preprocessing pipeline

Implemented in `src/features.py`:

- **Numeric:** StandardScaler on age, trestbps, chol, thalach, oldpeak
- **Categorical:** OneHotEncoder(handle_unknown="ignore") on sex, cp, fbs, restecg, exang, slope, ca, thal
- Wrapped with the estimator in an sklearn Pipeline so training and serving use identical transforms

5.2 Models trained (≥ 2 required)

Model	Rationale
Logistic Regression	Strong linear baseline, calibrated probabilities, interpretable coefficients
Random Forest	Non-linear interactions, robust to feature scale
XGBoost	High-capacity gradient boosting for tabular data

5.3 Tuning & validation

- Stratified train/test split (80/20, random_state=42)
- GridSearchCV with **5-fold StratifiedKFold**, scoring=roc_auc
- Final comparison on held-out test metrics: Accuracy, Precision, Recall, F1, ROC-AUC
- Leaderboard CSV: `artifacts/metrics/model_leaderboard.csv`

Hyperparameter grids (summary):

- **LogReg**: $C \in \{0.1, 1, 10\}$
- **RF**: $n_estimators \in \{100, 200\}$, $max_depth \in \{None, 5, 10\}$, $min_samples_split \in \{2, 5\}$
- **XGB**: $n_estimators \in \{100, 200\}$, $max_depth \in \{3, 5\}$, $learning_rate \in \{0.05, 0.1\}$

5.4 Model selection

The model with the highest **test ROC-AUC** is serialized to:

```
models/heart_disease_pipeline.joblib
models/model_metadata.json
```

Selection rationale is logged in MLflow run `best_model_selection`.

6. Experiment Tracking (MLflow)

Tracking URI: `./mlflow_tracking` (local file store).

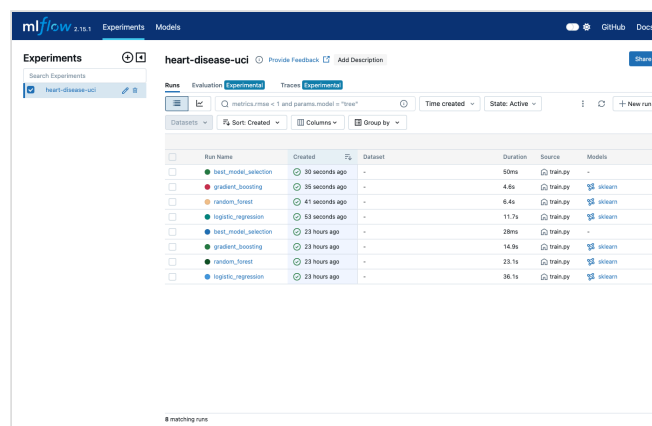
For each model run we log:

- Parameters (best GridSearch hyperparameters, model name, split seed)
- Metrics (accuracy, precision, recall, F1, ROC-AUC, CV ROC-AUC mean/std)
- Artifacts (confusion matrix PNG, ROC curve PNG, sklearn model)

Launch UI:

```
mlflow ui --backend-store-uri ./mlflow_tracking --port 5000
```

MLflow experiment tracking



The screenshot shows the MLflow UI interface for tracking experiments. The top navigation bar includes 'Experiments' and 'Models'. The main content area displays a table of runs for the 'heart-disease-uci' experiment. The table columns are Run Name, Created, F1, Dataset, Duration, Source, and Models. The runs listed are: best_model_selection, gradient_boosting, random_forest, logistic_regression, best_model_selection, gradient_boosting, random_forest, and logistic_regression. The 'best_model_selection' run is highlighted as the best model.

Run Name	Created	F1	Dataset	Duration	Source	Models
best_model_selection	30 seconds ago	-	-	50ms	train.py	-
gradient_boosting	35 seconds ago	-	-	4.6s	train.py	sklearn
random_forest	41 seconds ago	-	-	6.4s	train.py	sklearn
logistic_regression	53 seconds ago	-	-	11.7s	train.py	sklearn
best_model_selection	23 hours ago	-	-	28ms	train.py	-
gradient_boosting	23 hours ago	-	-	14.9s	train.py	sklearn
random_forest	23 hours ago	-	-	23.1s	train.py	sklearn
logistic_regression	23 hours ago	-	-	36.1s	train.py	sklearn

MLflow UI — heart-disease-uci runs

7. Model Packaging & Reproducibility

Concern	Approach
Environment	Pinned requirements.txt
Preprocessing	Saved inside the joblib Pipeline
Model binary	joblib (MLflow also stores sklearn flavor)
Metadata	model_metadata.json (best model name, AUC, feature order)

Concern	Approach
Data	Download script + cleaned CSV

Inference never re-implements ad-hoc scaling; it loads the full pipeline.

8. CI/CD Pipeline & Automated Testing

8.1 Tests (tests/)

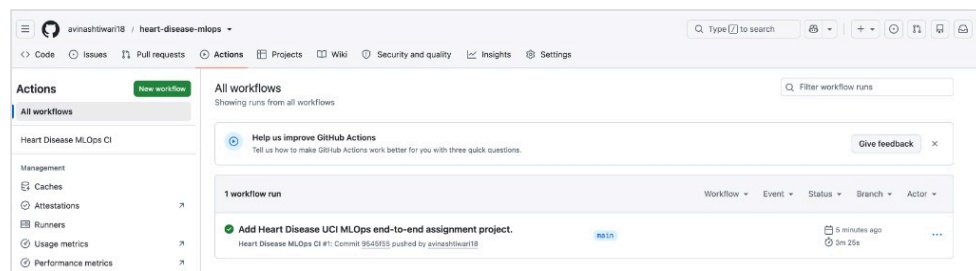
- `test_data_processing.py` – schema, imputation, split, outlier clipping
- `test_model.py` – preprocessor shape, pipeline fit/predict, saved model smoke test
- `test_api.py` – `/health`, `/predict` contract, validation errors, `/metrics`

8.2 GitHub Actions (`.github/workflows/ci.yml`)

Stages: checkout → Python 3.11 → install → flake8 → download data → pytest → EDA → train → full pytest → upload artifacts → Docker build → container `/predict` smoke test.

Failure policy: critical flake8 errors or any pytest failure fail the workflow; logs are visible in the Actions UI.

CI/CD evidence



GitHub Actions green run

9. Model Containerization

Dockerfile packages:

- Python 3.11 runtime
- API code (`api/`, `src/`)
- Trained model (`models/`)
- Dependencies from `requirements.txt`

Exposes port 8000 with healthcheck on `/health`.

```
docker build -t heart-disease-api:1.0.0 .
docker run --rm -p 8000:8000 heart-disease-api:1.0.0
bash scripts/sample_predict.sh
```

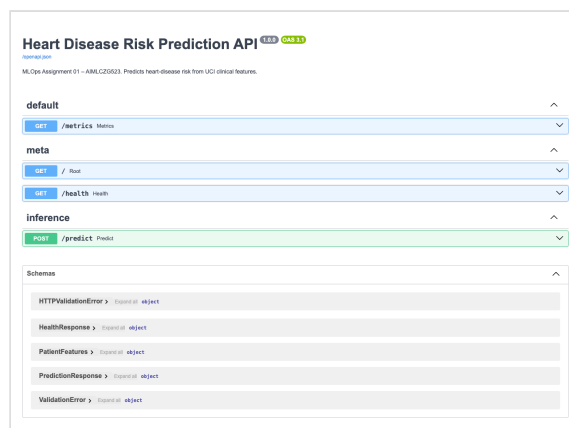
FastAPI Swagger UI: `http://127.0.0.1:8000/docs`

Docker serving evidence

```
Docker: heart-disease-api /health • /predict

$ docker build -t heart-disease-api:1.0.0
# Build succeeded
$ docker run -d --name hd-api -p 8000:8000 heart-disease-api:1.0.0
$ curl -s http://127.0.0.1:8000/health | python -m json.tool
{
  "status": "ok",
  "model_loaded": true,
  "model_name": "logistic_regression"
}
$ curl -s -X POST http://127.0.0.1:8000/predict -H 'Content-Type: application/json' -d '{...}'
{
  "prediction": 0,
  "prediction_label": "low_risk",
  "confidence": 0.8148,
  "probability_disease": 0.1852,
  "model_name": "logistic_regression"
}
```

Docker /health and /predict



FastAPI Swagger UI

10. Production Deployment (Kubernetes)

Manifests:

- k8s/deployment.yaml – Deployment (2 replicas) + LoadBalancer Service
- k8s/service.yaml – NodePort 30080 for local clusters
- k8s/ingress.yaml – optional Ingress host heart-disease.local
- k8s/helm/heart-disease-api/ – Helm chart alternative

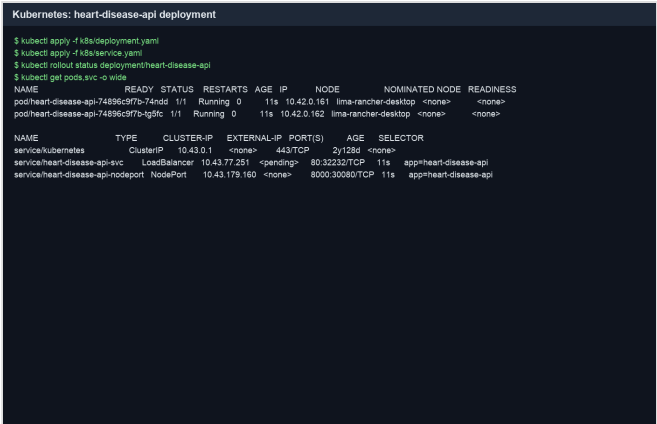
Local deploy:

```
bash scripts/deploy_minikube.sh
kubectl port-forward svc/heart-disease-api-svc 8000:80
```

Verification:

```
kubectl get pods,svc
bash scripts/sample_predict.sh http://127.0.0.1:8000
```

Kubernetes deployment evidence



Kubernetes pods and services

Access instructions (local testing)

- 1. Start Minikube / enable Kubernetes in Docker Desktop
- 2. Run scripts/deploy_minikube.sh
- 3. Port-forward service to localhost:8000
- 4. Use Swagger /docs or scripts/sample_predict.sh

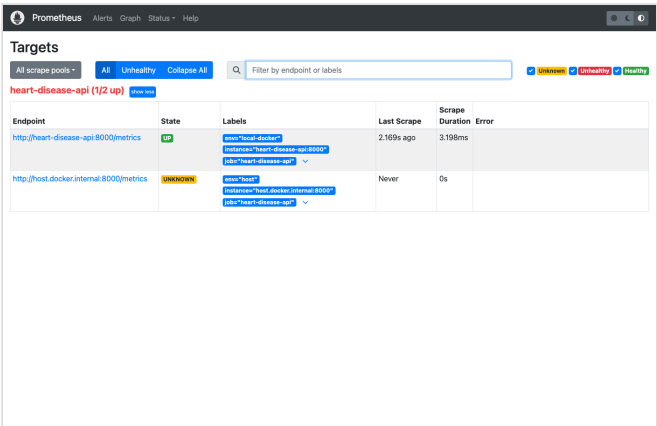
If a public cloud URL is later provisioned (GKE/EKS/AKS/Render), replace this section with the LoadBalancer external IP / Ingress hostname.

11. Monitoring & Logging

Signal	Implementation
Request logs	Middleware logs method, path, status, latency; file sink artifacts/logs/api_requests.log
Metrics	prometheus-fastapi-instrumentator exposes /metrics
Dashboards	Prometheus + Grafana via docker compose --profile monitoring up or k8s/monitoring/

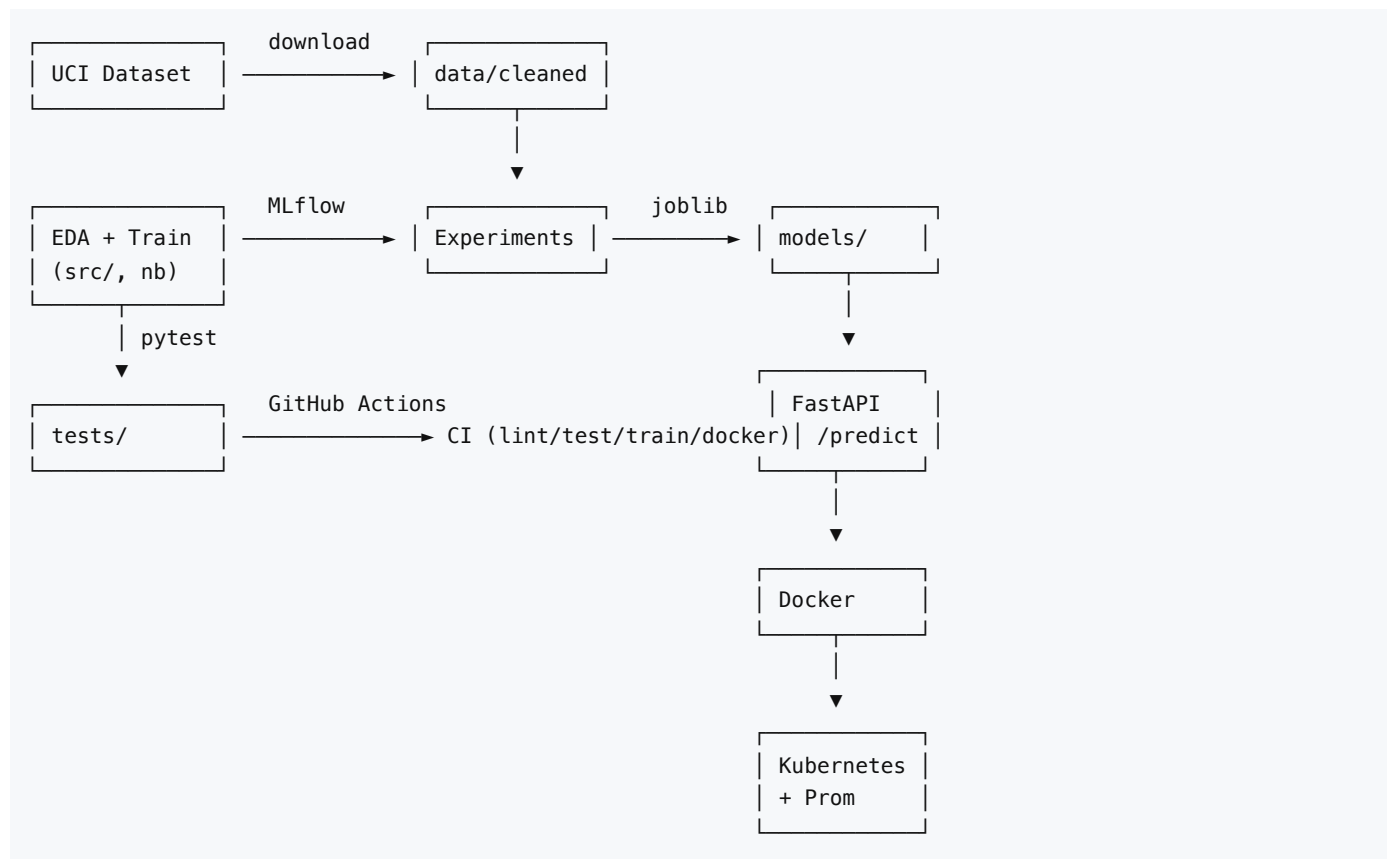
Why monitoring matters for ML systems: detect API downtime, latency regressions, elevated 5xx rates, and (with future extensions) prediction drift / data drift.

Monitoring evidence

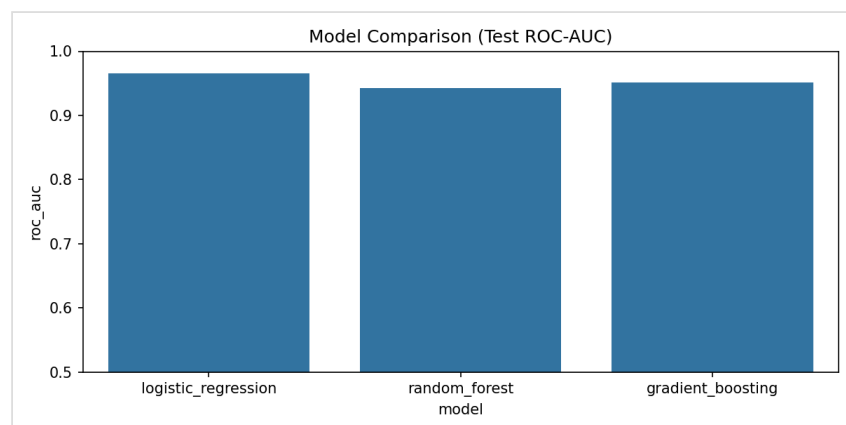


Prometheus targets UI

12. Solution Architecture



13. Results Summary



Model comparison ROC-AUC

Held-out test metrics from the local training run (python -m src.train):

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC (mean ± std)
logistic_regression	0.885	0.839	0.929	0.881	0.965	0.903 ± 0.016
random_forest	0.869	0.833	0.893	0.862	0.943	0.899 ± 0.031
gradient_boosting*	0.885	0.839	0.929	0.881	0.951	0.875 ± 0.030

*XGBoost is used when native libs are available (Linux CI / brew install libomp on macOS); otherwise sklearn GradientBoostingClassifier is trained as the third model.

Selected model: logistic_regression (highest test ROC-AUC = 0.965)

Best params: C=1.0, penalty=l2, solver=lbfgs

Key observation: logistic regression outperformed the ensembles on this relatively small, well-structured clinical dataset, delivering strong discrimination (ROC-AUC ≈ 0.97) with a simpler, more interpretable model—suitable for a monitored production API.

14. Challenges & Lessons Learned

1. **Schema inconsistency across mirrors** – Cleveland uses num + ? missings; some CSV mirrors already expose binary target. The loader normalizes both.
 2. **Inference parity** – Saving only a bare estimator would break production; packaging the full Pipeline is mandatory.
 3. **CI hermeticity** – The workflow downloads data and trains from scratch so runners do not depend on local binaries.
 4. **Local K8s image pull** – Minikube requires building inside its Docker daemon (minikube docker-env) when using IfNotPresent.
-

15. Future Improvements

- Add data-drift monitors (evidently / custom PSI)
 - Model registry stage transitions (Staging → Production) in MLflow
 - Canary deployment with Istio / Flagger
 - Calibrated probability thresholds tuned for clinical recall targets
 - Public cloud deploy with HTTPS Ingress and secrets management
-

16. Deliverables Mapping

Assignment requirement	Location
Data acquisition & EDA	scripts/download_data.py, src/eda.py, notebooks/01_eda.ipynb, artifacts/eda/
Feature eng. & models	src/features.py, src/train.py, notebooks/02_training.ipynb
Experiment tracking	MLflow in src/train.py, store mlflow_tracking/
Packaging	models/*.joblib, requirements.txt
CI/CD & tests	.github/workflows/ci.yml, tests/
Containerization	Dockerfile, docker- compose.yml
Deployment	k8s/, scripts/deploy_minikube.sh
Monitoring	API logging, /metrics, k8s/monitoring/
Documentation	README.md, this REPORT.md, screenshots/

17. References

1. UCI Machine Learning Repository – Heart Disease Dataset
 2. MLflow Documentation – <https://mlflow.org/docs/latest/index.html>
 3. FastAPI Documentation – <https://fastapi.tiangolo.com/>
 4. Kubernetes Documentation – <https://kubernetes.io/docs/home/>
 5. Prometheus – <https://prometheus.io/docs/>
-

Appendix A – Example API response

```
{
  "prediction": 1,
  "prediction_label": "heart_disease_risk",
  "confidence": 0.82,
  "probability_disease": 0.82,
  "model_name": "random_forest"
}
```

Appendix B – Video demo outline

Record a 3–5 minute screencast covering: repo walkthrough → EDA plots → MLflow UI → pytest/CI → Docker predict → Kubernetes pods → Prometheus metrics.

End of report. Export this Markdown to PDF/DOCX (e.g., Pandoc, Word, or VS Code Markdown PDF) and attach screenshots to reach the ~10-page submission length.